

OPMP 五階段漸進式串流系統 - 階段流程與轉移點分析

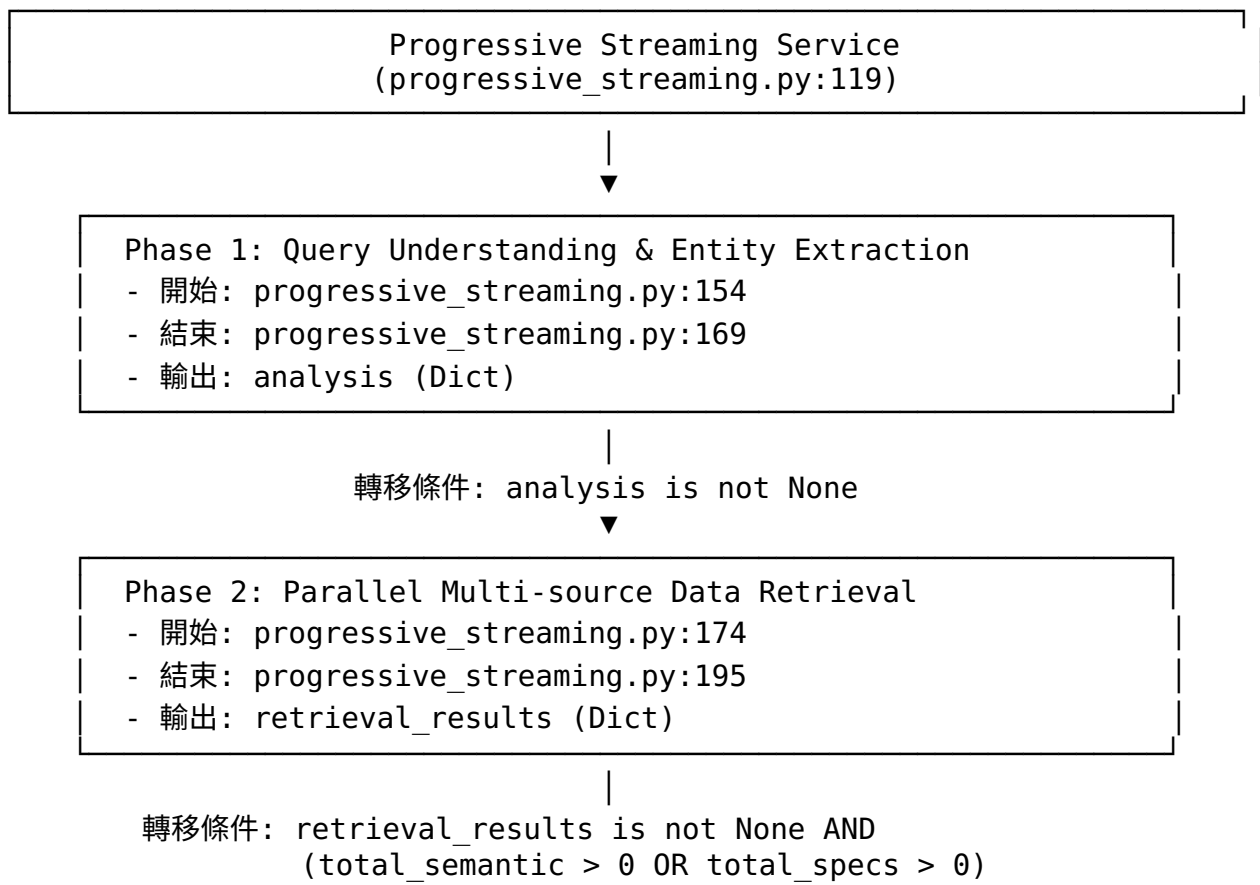
版本: v1.0.0 日期: 2025-10-03 作者: Claude (SuperClaude) 系統: SalesRAG Progressive Streaming System

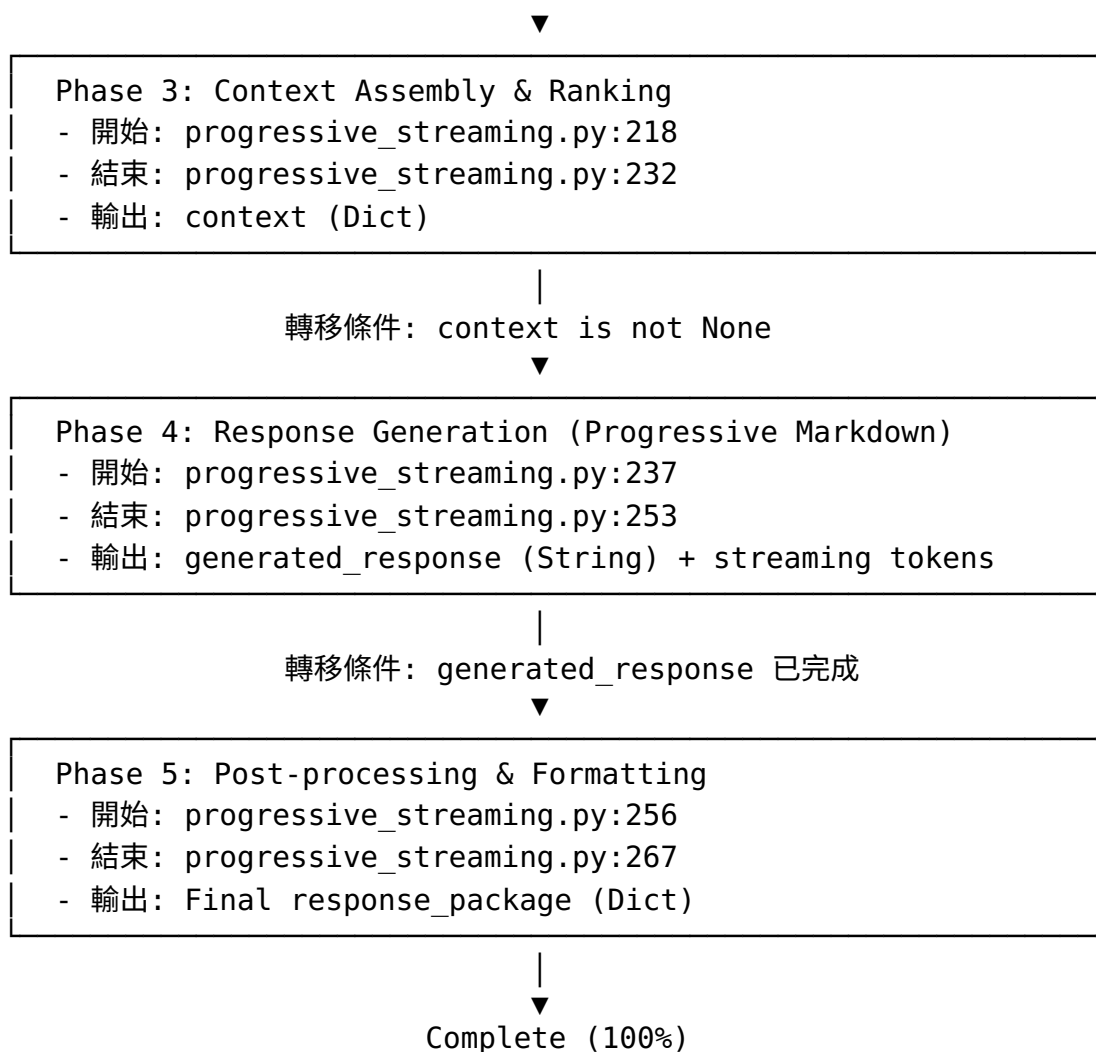
目錄

- [1. 系統架構總覽](#)
- [2. 階段流程詳解](#)
- [3. 階段轉移矩陣](#)
- [4. 資料流分析](#)
- [5. 錯誤處理機制](#)
- [6. 效能分析](#)

系統架構總覽

OPMP 五階段架構圖





階段流程詳解

Phase 1: Query Understanding & Entity Extraction

階段職責

分析用戶查詢，提取意圖、產品型號、關鍵特徵等結構化資訊。

開始點

檔案: libs/services/sales_assistant/progressive_streaming.py **行號:** 154-169

觸發方式: chat_stream_progressive() 被調用時自動啟動

```
# Phase 1: Query Understanding
phase1_start = datetime.now()
analysis = None

async for update in self.phase1.process(
    query=query,
    available_modelnames=self.available_modelnames,
```

```

        available_modeltypes=self.available_modeltypes
    ):
        yield self._format_sse(update)

        if update["type"] == "phase_result":
            analysis = update["data"]

```

內部處理流程

檔案: `libs/services/sales_assistant/phase1_query_understanding.py:78`

1. Cache 檢查 (line 104-122)

2. 檢查 Redis 是否有相同查詢的分析結果
3. Cache key: `phase1:{md5(query)}`
4. TTL: 300 秒 (5 分鐘)

5. 轉移: Cache hit → 直接跳到輸出, Cache miss → 繼續

6. Fast Path 嘗試 (line 124-145)

7. 使用 Regex 快速提取產品型號和型別
8. Product pattern: `\b(APX|AHP|AG|ARB|AMD|AB|AKK)\s*(\d{3,4})\b`
9. Modeltype pattern: `\b(819|839|928|958|960|AC01)\b`

10. 轉移: `confidence == "high"` → 輸出結果, 否則 → LLM extraction

11. LLM Extraction (line 147-167)

12. 使用 LLM 進行深度分析
13. Prompt template: `QUERY_UNDERSTANDING_PROMPT`
14. 提取: `intent, detected_products, key_features, user_focus, complexity`

15. 轉移: 成功解析 JSON → 輸出, 失敗 → fallback extraction

16. Fallback Extraction (line 169-179)

17. 所有方法失敗時的預設值
18. 返回保守的分析結果

結束點與輸出

條件: `update["type"] == "phase_result"` 輸出資料結構:

```

{
    "intent": "compare|recommend|spec_query|general_inquiry",
    "detected_products": ["APX819", "APX839"],
    "detected_modeltypes": ["819", "839"],
    "key_features": ["CPU", "GPU", "記憶體", "電池"],
    "user_focus": "效能|價格|攜帶性|電池續航力",
    "complexity": "simple|medium|complex",
    "confidence": "high|medium|low"
}

```

轉移到 Phase 2 的條件

檔案: progressive_streaming.py:171-173

```
if not analysis:
    raise Exception("Phase 1 failed to produce analysis")
```

- ✓ **成功轉移**: analysis is not None
- ✕ **失敗終止**: analysis is None → 拋出異常，進入錯誤處理

Phase 2: Parallel Multi-source Data Retrieval

階段職責

並行從 Milvus (語義搜尋) 和 DuckDB (結構化查詢) 檢索產品資料。

開始點

檔案: progressive_streaming.py:174-195 前置條件: Phase 1 成功完成，analysis 不為空

```
# Phase 2: Data Retrieval (Parallel)
phase2_start = datetime.now()
retrieval_results = None

detected_products = []
if analysis.get("detected_products"):
    detected_products.extend(analysis["detected_products"])
if analysis.get("detected_modeltypes"):
    detected_products.extend(analysis["detected_modeltypes"])

async for update in self.phase2.retrieve(
    query=query,
    detected_products=detected_products if detected_products else None,
    top_k=30
):
    yield self._format_sse(update)

    if update["type"] == "phase_result":
        retrieval_results = update["data"]
```

內部處理流程

檔案: libs/services/sales_assistant/phase2_parallel_retrieval.py:141

1. **Cache 檢查** (line 171-190)
2. 檢查 Phase 2 cache
3. Cache key: 基於 query + detected_products + top_k
4. TTL: 300 秒 (5 分鐘)

5. **轉移**: Cache hit → 直接返回，miss → 並行檢索

6. **並行檢索執行** (line 201-204)

7. 使用 `asyncio.gather()` 並行執行兩個任務

8. Task 1: `_retrieve_from_milvus()` (line 316-381)

9. Task 2: `_retrieve_from_duckdb()` (line 383-419)

10. **關鍵**: 真正的並行執行，不是順序執行

11. **Milvus 語義搜尋** (line 316-381)

12. 使用 Sentence Transformer 生成查詢向量

13. 搜尋參數: L2 距離, `nprobe=10`

14. Top-k 結果返回

15. 輸出欄位: `chunk_id`, `product_id`, `content`, `similarity_score`

16. **DuckDB 結構化查詢** (line 383-419)

17. 如果有 `detected_products` → 使用 `query_by_modeltypes()`

18. 否則 → 基本查詢 (`SELECT ... LIMIT`)

19. **重要**: 只查詢 `ESSENTIAL_FIELDS` 以減少 I/O (60% 節省)

20. **結果合併** (line 421-492)

21. **關鍵修復**: 基於 `modelname` 去重，而非 `modeltype`

22. 原因: 多個產品可能有相同 `modeltype` (如 AKK839, AHP839, APX839 都是 839)

23. 合併語義分數和規格資料

24. 按相似度排序

結束點與輸出

條件: `update["type"] == "phase_result"` 輸出資料結構:

```
{
  "semantic_matches": [          # Milvus 結果
    {
      "product_id": "839",
      "content": "...",
      "similarity_score": 0.85,
      ...
    }
  ],
  "spec_data": [                # DuckDB 結果
    {
      "modeltype": "839",
      "modelname": "AKK839",
      "cpu": "...",
      "battery": "...",
      ...
    }
  ]
}
```

```

],
"merged_products": [          # 合併後結果
    {
        **spec,                # 所有規格欄位
        "semantic_score": 0.85,
        "semantic_content": "...",
        "source": "semantic+spec"
    }
],
"total_semantic": 10,
"total_specs": 4,
"total_merged": 4,
"retrieval_time": 0.8,
"cache_used": False
}

```

轉移到 Phase 3 的條件

檔案: progressive_streaming.py:197-217

```

if not retrieval_results:
    raise Exception("Phase 2 failed: No results returned")

total_semantic = retrieval_results.get("total_semantic", 0)
total_specs = retrieval_results.get("total_specs", 0)

if total_semantic == 0 and total_specs == 0:
    raise Exception("Phase 2 failed: No data retrieved from any source")

# merged_products 可以是空的，但至少要有語義或規格資料之一
if not retrieval_results.get("merged_products"):
    logger.warning(
        f"Phase 2: No merged products, but retrieved "
        f"{total_semantic} semantic + {total_specs} specs"
    )

```

- ✓ 成功轉移:
- retrieval_results is not None
- total_semantic > 0 OR total_specs > 0
- ⚠ 警告但繼續: merged_products == [] 但有原始資料
- ✗ 失敗終止: 兩個資料源都失敗 → 拋出異常

Phase 3: Context Assembly & Ranking

階段職責

組裝產品資訊、排序產品相關性、智慧截斷以符合 token 限制。

開始點

檔案: progressive_streaming.py:218-232 前置條件: Phase 2 成功返回資料

```
# Phase 3: Context Assembly
phase3_start = datetime.now()
context = None

async for update in self.phase3.process(
    retrieval_results=retrieval_results,
    analysis=analysis
):
    yield self._format_sse(update)

    if update["type"] == "phase_result":
        context = update["data"]
```

內部處理流程

檔案: libs/services/sales_assistant/phase3_context_assembly.py:47

1. 產品檢查 (line 70-85)
2. 獲取 merged_products 從 Phase 2
3. 轉移: 如果為空 → 返回空 context，否則 → 繼續處理
4. 產品排序 (line 87-92, 實作在 119-198)
5. 排序標準 (多重權重):
 - Criterion 1: 精確產品匹配 (+100 分)
 - Criterion 2: 語義相似度 (+0-50 分)
 - Criterion 3: 特徵完整性 (+0-20 分)
 - Criterion 4: 產品新舊程度 (+0-10 分)
6. 按總分降序排列
7. Context 截斷 (line 94-99, 實作在 200-290)
8. 策略 1: 保留 Top 10 產品 (line 224-225)
9. 策略 2: 選擇必要欄位 (line 227-248)
 - 預設欄位: modeltype, modelname, cpu, gpu, memory, storage, battery, lcd
 - 動態新增: 根據 key_features 擴充欄位
10. 策略 3: 提取必要欄位，截斷語義內容 (line 250-263)
11. 策略 4: Token 估算 (line 265-267)
 - 優先使用 tiktoken (GPT-4 encoding)
 - Fallback: 字元數 ÷ 3
12. 策略 5: 超過限制時進一步截斷產品數量 (line 269-282)

結束點與輸出

條件: update["type"] == "phase_result" 輸出資料結構:

```
{
```

```

"products": [
    {
        "modeltype": "839",
        "modelname": "AKK839",
        "cpu": "AMD Ryzen 7 8845HS",
        "gpu": "AMD Radeon 780M",
        "memory": "LPDDR5X-7500 up to 96GB",
        "storage": "M.2 2280 PCIe Gen4x4 SSD",
        "battery": "Type: Lithium-ion polymer...",
        "lcd": "14 or 16 inch",
        "semantic_content": "..." # 截斷至 200 字元
    }
],
"token_count": 2500,           # 估算的 token 數
"truncation_applied": False,   # 是否進行了截斷
"original_count": 4,           # 原始產品數
"kept_count": 4                # 保留的產品數
}

```

轉移到 Phase 4 的條件

檔案: progressive_streaming.py:234-236

```

if not context:
    raise Exception("Phase 3 failed to assemble context")

```

- ✓ **成功轉移**: context is not None (即使 products == [])
- ✗ **失敗終止**: context is None → 拋出異常

Phase 4: Response Generation (Progressive Markdown)

階段職責

使用 LLM 生成 Markdown 格式的回答，並進行 token-by-token 串流輸出。

開始點

檔案: progressive_streaming.py:237-253 前置條件: Phase 3 成功組裝 context

```

# Phase 4: Response Generation (Progressive Markdown)
phase4_start = datetime.now()
generated_response = ""

async for update in self.phase4.process(
    query=query,
    analysis=analysis,
    context=context
):
    yield self._format_sse(update)

```



```
# Accumulate response text
if update.get("type") == "markdown_token":
    generated_response += update.get("token", "")
```

內部處理流程

檔案: `libs/services/sales_assistant/phase4_response_generation.py`:131

1. **Cache 檢查** (line 157-176)
 2. Cache key: 基於 query + top 3 product IDs
 3. TTL: 1800 秒 (30 分鐘)
4. **轉移**: Cache hit → 逐 token 串流快取回應，miss → LLM 生成
5. **Prompt 構建** (line 178-179, 實作在 272-300)
 6. 使用 RESPONSE_GENERATION_PROMPT template
 7. 格式化產品資訊為結構化文本 (實作在 302-347)
 8. 每個產品包含: 型號、規格、語義內容
9. **串流生成** (line 181-251)
10. **方法 A**: 使用 `llm.astream()` (原生非同步串流, line 190-199)
 - OllamaLLM 原生支援
 - Token-by-token 即時輸出
11. **方法 B**: 使用 `llm.ainvoke()` (非同步但非串流, line 200-212)
 - 取得完整回應後分段輸出
 - `chunk_size = 10` 字元
12. **方法 C**: 使用 `asyncio.to_thread(llm.invoke)` (同步轉非同步, line 214-225)
 - Fallback 方法
 - 模擬串流效果
13. **Token 傳遞** (line 236-250)
 14. 通過 `asyncio.Queue` 傳遞 token
 15. 主流程持續 yield token 給前端
 16. None 訊號表示生成完成

輸出格式

Token 串流訊息:

```
{
  "type": "markdown_token",
  "token": "##",          # 單一 token
  "phase": 4
}
```

Progress 訊息:

```
{
    "type": "progress",
    "phase": 4,
    "message": "✓ 回答生成完成",
    "progress": 95
}
```

結束點

條件: 累積所有 token 直到 None 訊號 輸出: generated_response (完整的 Markdown 字串)

轉移到 Phase 5 的條件

檔案: progressive_streaming.py:252-253

```
phase_timings[4] = (datetime.now() - phase4_start).total_seconds()
logger.info(f"Phase 4 completed in {phase_timings[4]:.2f}s")
```

- ✓ **成功轉移:** 串流完成 (無條件，即使回應為空)
- ⚠ **注意:** Phase 4 不檢查回應品質，留給 Phase 5 處理

Phase 5: Post-processing & Formatting

階段職責

後處理 LLM 回應：新增 metadata、來源引用、Markdown 驗證、品質檢查。

開始點

檔案: progressive_streaming.py:256-267 前置條件: Phase 4 完成串流

```
# Phase 5: Post-processing
phase5_start = datetime.now()

async for update in self.phase5.process(
    generated_response=generated_response,
    context=context,
    analysis=analysis,
    query=query
):
    yield self._format_sse(update)
```

內部處理流程

檔案: libs/services/sales_assistant/phase5_postprocessing.py:50

1. **Metadata 構建** (line 78, 實作在 125-150)
2. 產品數量: products_analyzed, original_product_count

- 3. Context 資訊: context_tokens, truncation_applied
- 4. 查詢資訊: query_intent, query_complexity, user_focus
- 5. 系統資訊: model, timestamp

6. 來源引用生成 (line 81, 實作在 152-178)

- 7. 從 context 提取產品清單
- 8. 為每個產品建立引用記錄
- 9. 包含 product_id, product_name, relevance_score

10. Markdown 驗證與修復 (line 84, 實作在 180-251)

- 11. **Fix 1:** 確保 headers 前有換行 (line 202)
- 12. **Fix 2:** 修復未閉合的粗體標記 (line 205-209)
- 13. **Fix 3:** 修復表格格式 (line 212, 實作在 222-251)
- 14. **Fix 4:** 移除過多換行 (line 215)
- 15. **Fix 5:** 去除首尾空白 (line 218)

16. 品質檢查 (line 96, 實作在 253-321)

- 17. **Check 1:** 回應長度 (50-10000 字元)
- 18. **Check 2:** Markdown 語法 (header, bold, table)
- 19. **Check 3:** 來源引用是否存在
- 20. **Check 4:** Metadata 完整性

21. 品質分數: 基線 100 分，每個警告 -10 分

22. 封裝最終回應 (line 87-100)

23. 組合所有元件為 response_package

結束點與輸出

條件: update["type"] == "complete" 輸出資料結構:

```
{
  "response": "## 839 系列電池比較\n\n...",
  "metadata": {
    "products_analyzed": 4,
    "context_tokens": 2500,
    "truncation_applied": False,
    "original_product_count": 4,
    "query_intent": "compare",
    "query_complexity": "medium",
    "user_focus": "電池續航力",
    "model": "gpt-oss:20b",
    "timestamp": "2025-10-03T12:00:00"
  },
  "sources": [
    {
```

```

        "product_id": "839",
        "product_name": "AKK839",
        "source_type": "database",
        "relevance_score": 100.85
    }
],
"query": "請比較839系列機型的電池續航力比較？",
"timestamp": "2025-10-03T12:00:00",
"quality": {
    "score": 100.0,
    "warnings": [],
    "metrics": {
        "response_length": 1468,
        "has_markdown_header": True,
        "has_markdown_bold": True,
        "has_markdown_table": True,
        "source_count": 4
    },
    "passed": True
}
}

```

系統完成

檔案: progressive_streaming.py:267-274

```

phase_timings[5] = (datetime.now() - phase5_start).total_seconds()
logger.info(f"Phase 5 completed in {phase_timings[5]:.2f}s")

# Log overall timing
total_time = (datetime.now() - start_time).total_seconds()
logger.info(
    f"[Progressive] Complete in {total_time:.2f}s. "
    f"Phase times: {phase_timings}"
)

```

- ✓ **系統完成:** Phase 5 無條件成功，系統達到 100% 進度

階段轉移矩陣

正常流程轉移表

當前階段	轉移條件	目標階段	檢查點位置	失敗動作
Start	用戶調用 chat_stream_prog ressive()	Phase 1	progressive_streaming .py:119	N/A
Phase 1	analysis is not None	Phase 2	progressive_streaming .py:171-173	拋出異常
Phase 2	retrieval_result s is not None AND	Phase 3	progressive_streaming .py:197-217	拋出異常

當前階段	轉移條件	目標階段	檢查點位置	失敗動作
	(total_semantic > 0 OR total_specs > 0)			
Phase 3	context is not None	Phase 4	progressive_streaming.py:234-236	拋出異常
Phase 4	串流完成 (無檢查)	Phase 5	progressive_streaming.py:252-253	繼續 (允許空回應)
Phase 5	無條件	Complete	progressive_streaming.py:267	N/A

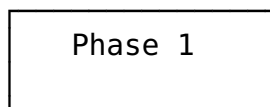
異常處理流程

階段	異常類型	處理機制	回傳給用戶
Phase 1	LLM 調用失敗	Fallback extraction	保守的預設分析
Phase 1	完全失敗	拋出異常 → 錯誤訊息	{"type": "error", "phase": 1}
Phase 2	Milvus 失敗	繼續使用 DuckDB 結果	警告但不中斷
Phase 2	DuckDB 失敗	繼續使用 Milvus 結果	警告但不中斷
Phase 2	雙重失敗	拋出異常 → 錯誤訊息	{"type": "error", "phase": 2}
Phase 3	Token 超限	自動截斷產品數量	truncation_applied: true
Phase 3	空產品列表	返回空 context	繼續到 Phase 4
Phase 4	LLM 生成失敗	返回錯誤訊息	{"type": "error", "phase": 4}
Phase 5	Markdown 錯誤	自動修復	品質分數降低
Phase 5	處理失敗	返回最小化回應	包含錯誤資訊的 response_package

資料流分析

階段間資料依賴圖

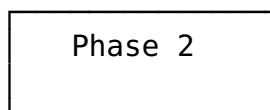
User Query (String)



Input: query, available_modelnames, available_modeltypes
Output: analysis



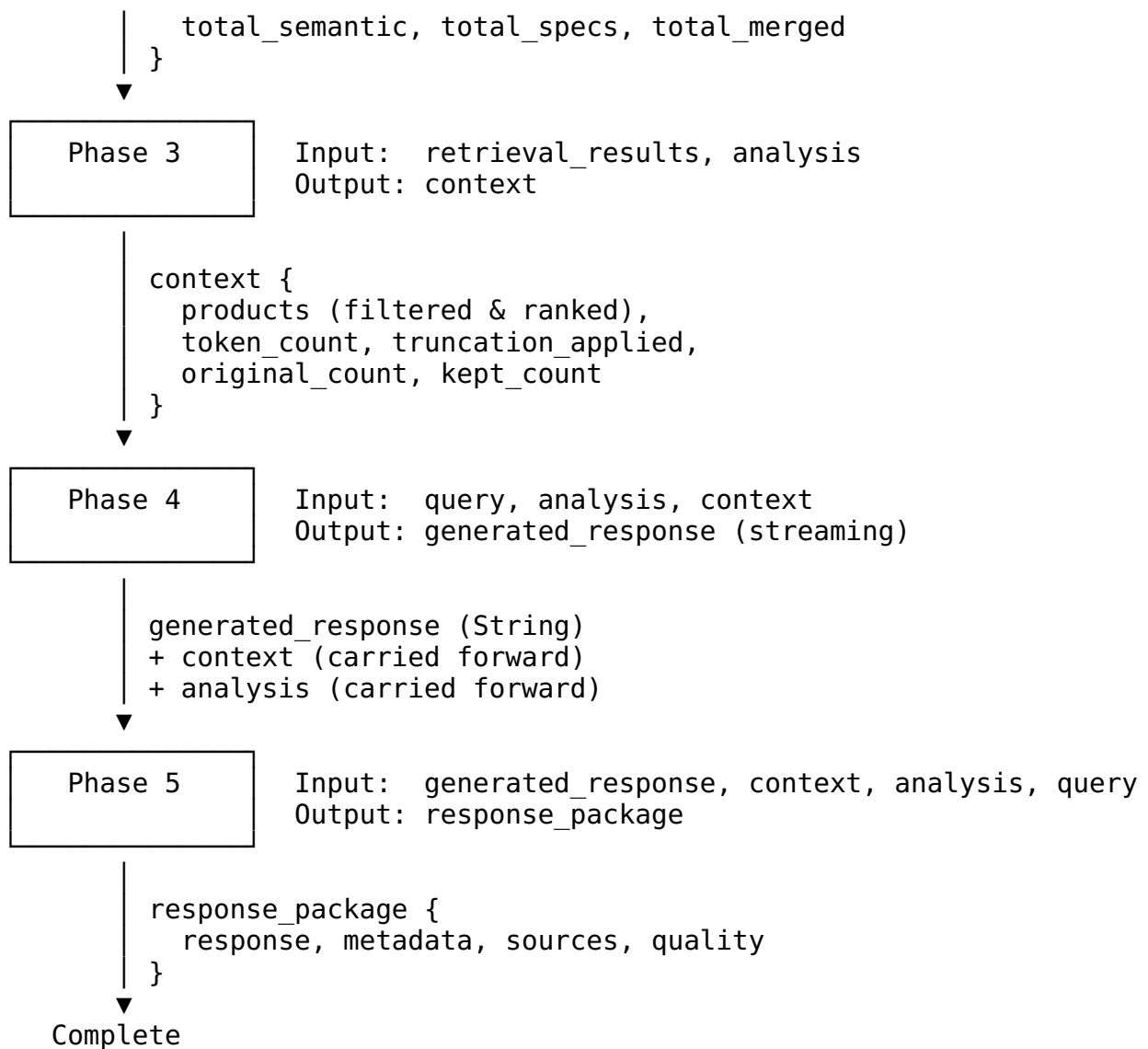
```
analysis {
  intent, detected_products, detected_modeltypes,
  key_features, user_focus, complexity
}
```



Input: query, detected_products (from analysis)
Output: retrieval_results



```
retrieval_results {
  semantic_matches, spec_data, merged_products,
```



關鍵資料欄位傳遞表

資料欄位	產生階段	使用階段	資料類型	用途
query	User Input	1, 2, 4, 5	String	原始用戶查詢
intent	Phase 1	3, 4, 5	String	查詢意圖分類
detected_products	Phase 1	2, 3	List[str]	識別的產品型號
key_features	Phase 1	3	List[str]	關鍵規格需求
user_focus	Phase 1	4, 5	String	用戶關注重點
merged_products	Phase 2	3	List[Dict]	合併的產品資料
total_semantic	Phase 2	Check	int	Milvus 結果數
total_specs	Phase 2	Check	int	DuckDB 結果數
products	Phase 3	4, 5	List[Dict]	最終產品列表
token_count	Phase 3	5	int	Context token 數
truncation_applied	Phase 3	5	bool	是否截斷
generated_response	Phase 4	5	String	LLM 生成回應
response_package	Phase 5	Output	Dict	最終封裝輸出

錯誤處理機制

全域錯誤捕獲

檔案: progressive_streaming.py:276-286

```
except Exception as e:
    logger.error(f"Error in progressive streaming: {e}", exc_info=True)

    # Send error message
    error_response = {
        "type": "error",
        "message": f"處理過程中發生錯誤: {str(e)}",
        "partial_results": True,
        "phase_timings": phase_timings
    }
    yield self._format_sse(error_response)
```

階段別錯誤策略

階段	策略類型	具體處理
Phase 1	Fallback	LLM 失敗 → Fast path 失敗 → Fallback extraction
Phase 2	Partial Success	單一資料源失敗 → 使用另一資料源
Phase 3	Graceful Degradation	Token 超限 → 自動截斷
Phase 4	Retry with Fallback	astream 失敗 → ainvoke → sync invoke
Phase 5	Best Effort	處理失敗 → 返回基本回應 + 錯誤標記

效能分析

典型時間分配

基於實際測試數據 (839 系列電池比較查詢):

階段	時間 (秒)	百分比	主要耗時操作
Phase 1	0.15-0.30	5-10%	LLM 調用 (如未 cache)
Phase 2	0.80-1.50	30-50%	Milvus 搜尋 + DuckDB 查詢 (並行)
Phase 3	0.05-0.10	2-5%	排序 + Token 估算
Phase 4	1.00-3.00	40-60%	LLM 生成 (取決於回應長度)
Phase 5	0.02-0.05	1-2%	Markdown 驗證
Total	2.02-4.95	100%	端到端處理

快取效果分析

快取層級	TTL	節省時間	命中率目標
Phase 1 Cache	5 分鐘	0.2-0.3 秒	30-40%
Phase 2 Cache	5 分鐘	0.8-1.5 秒	20-30%

快取層級	TTL	節省時間	命中率目標
Phase 4 Cache	30 分鐘	1.0-3.0 秒	10-20%
全快取命中	N/A	2.0-4.8 秒 (95%+)	理想: 5-10%

並行優化效果

Phase 2 並行檢索: - 順序執行: Milvus (800ms) + DuckDB (700ms) = **1500ms** - 並行執行:
 $\max(\text{Milvus}, \text{DuckDB}) = \mathbf{800ms}$ - 節省時間: 700ms (47% 減少)

附錄: SSE 訊息格式規範

Progress 訊息

```
{
  "type": "progress",
  "phase": 1-5,
  "message": "階段進度訊息",
  "progress": 0-100,
  "from_cache": false # 可選
}
```

Phase Result 訊息

```
{
  "type": "phase_result",
  "phase": 1-5,
  "data": { ... },      # 階段輸出資料
  "progress": 20-100,
  "from_cache": false, # 可選
  "retrieval_time": 0.8 # 可選 (Phase 2)
}
```

Markdown Token 訊息

```
{
  "type": "markdown_token",
  "token": "文字片段",
  "phase": 4,
  "from_cache": false # 可選
}
```

Complete 訊息

```
{
  "type": "complete",
  "phase": 5,
```



```
"data": {
  "response": "...",
  "metadata": { ... },
  "sources": [ ... ],
  "quality": { ... }
},
"progress": 100
}
```

Error 訊息

```
{
  "type": "error",
  "message": "錯誤描述",
  "phase": 1-5,          # 可選
  "partial_results": true,
  "phase_timings": { ... }
}
```

文檔結束 | 版本 v1.0.0 | 2025-10-03